

Deep Learning I

Basics, Convolutional Networks, and Sequence Modeling

Chengwei QIN

AI Thrust, Information Hub

2025.11.13

Today's Agenda

1 Deep Learning Basics:

- What are the building blocks? (Layers, Activations)
- How do they learn? (Loss, Optimization, Backpropagation)
- How do we make them learn *well*? (Initialization, Regularization, Normalization)

2 Convolutional Networks (CNNs):

- Why MLPs fail for images.
- How Convolutions and Pooling solve this.
- How to build deep CNNs (ResNets).

3 Sequence Modeling (RNNs):

- Why MLPs/CNNs fail for text.
- How RNNs use "memory" to read sequences.
- How LSTMs/GRUs solve the long-term memory problem.

Deep Learning Basics

Why "Deep" Learning?

- **Shallow Model (e.g., Logistic Regression):**

- Learns a single, simple decision boundary.
- Input $\rightarrow$ 1 Layer $\rightarrow$ Output

- **Deep Model (Deep Neural Network):**

- Learns a **hierarchy of features**.
- Input $\rightarrow$ Layer 1 $\rightarrow$ Layer 2 $\rightarrow \dots \rightarrow$ Layer N $\rightarrow$ Output

- **Example (Image Recognition):**

- **Layer 1** learns $\rightarrow$ Edges & Corners
- **Layer 2** learns $\rightarrow$ Combinations of edges (e.g., "eye", "nose", "wheel")
- **Layer 3** learns $\rightarrow$ Combinations of features (e.g., "face", "car")

Key Idea

"Deep" refers to this **depth of layers**, which allows the model to learn a **hierarchy of complex concepts** automatically from data.

The Neuron

A neuron computes a **weighted combination** of inputs and then applies a **nonlinearity**. It acts as a simple **feature detector**.

Step 1: Affine Map (Linear + Bias)

- Inputs as a vector $\mathbf{x} \in \mathbb{R}^d$, weights $\mathbf{w} \in \mathbb{R}^d$, bias $b \in \mathbb{R}$.
- Compute the score:

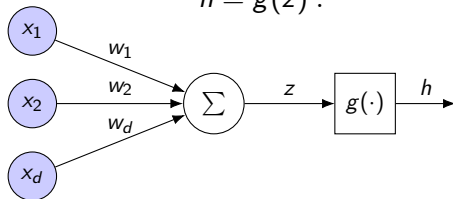
$$z = \mathbf{w}^\top \mathbf{x} + b .$$

- Geometrically: learns a separating **hyperplane**.

Step 2: Activation

- Apply a nonlinearity $g(\cdot)$ to enable **non-linear** decision boundaries:

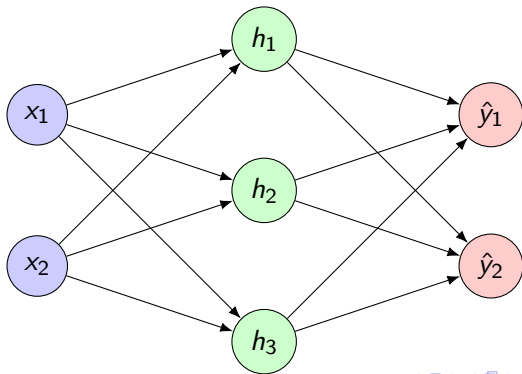
$$h = g(z) .$$



From Neurons to Layers: MLP

We stack neurons into **Layers**. A **Multi-Layer Perceptron (MLP)** is just a stack of these layers.

- A layer with M inputs and N outputs is a **fully-connected** or **linear** layer.
- It's just a matrix multiplication followed by an activation.
- **Input Layer** $\rightarrow$ **Hidden Layer(s)** $\rightarrow$ **Output Layer**



In PyTorch: nn.Linear

PyTorch's nn.Module system (Week 9) makes this easy.

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 class MLP(nn.Module):
5     def __init__(self, input_dim, hidden_dim, output_dim):
6         super().__init__()
7         # Define the layers (weights are created automatically)
8         self.fc1 = nn.Linear(input_dim, hidden_dim)
9         self.fc2 = nn.Linear(hidden_dim, hidden_dim)
10        self.fc3 = nn.Linear(hidden_dim, output_dim)
11    def forward(self, x):
12        # Define the computation flow
13        x = self.fc1(x)
14        x = F.relu(x)
15        x = self.fc2(x)
16        x = F.relu(x)
17        x = self.fc3(x)
18        return x
19 model = MLP(input_dim=784, hidden_dim=128, output_dim=10)
20 print(model)
```

The Role of Activation Functions

Question: What if we just stack linear layers?

$$\text{Layer 1: } y_1 = W_1x + b_1$$

$$\text{Layer 2: } y_2 = W_2y_1 + b_2$$

Substitute Layer 1 into Layer 2:

$$y_2 = W_2(W_1x + b_1) + b_2$$

$$y_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

$$y_2 = W_{\text{new}}x + b_{\text{new}}$$

Key Takeaway

A stack of linear layers is **one single linear layer**. The network cannot learn complex, non-linear patterns (like "cat" vs. "dog"). We **must** apply a **non-linear activation function** $g(\cdot)$ after each linear layer.

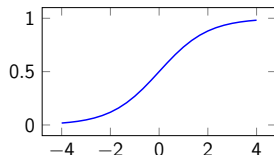
$$h = g(Wx + b)$$

Common Activations: Sigmoid, Tanh, ReLU

Sigmoid: $\frac{1}{1+e^{-x}}$

- Squashes to (0, 1)
- **Bad:** "Vanishing gradients" (gradient ≈ 0 for large/small x).
- **Bad:** Not zero-centered.

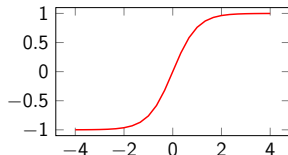
Sigmoid



Tanh: $\tanh(x)$

- Squashes to (-1, 1)
- **Good:** Zero-centered.
- **Bad:** Still has vanishing gradients.

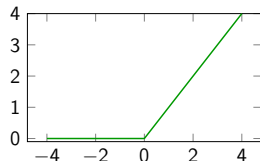
Tanh



ReLU: $\max(0, x)$

- **Good:** No vanishing gradient for $x > 0$.
- **Good:** Very fast.
- **Bad:** "Dying ReLU" (gradient is 0 for all $x < 0$).

ReLU



Why ReLU is the Default

- **Problem with Sigmoid/Tanh:** Look at the graph.
 - When the input x is large or small, the function becomes "saturated" (flat).
 - The slope (gradient) at these points is ≈ 0 .
 - During backpropagation, gradients are multiplied.
 $0.1 \times 0.1 \times 0.1 = 0.001$.
 - The signal to update the early layers "vanishes". This is the **Vanishing Gradient Problem**.
- **Solution with ReLU:**
 - For all positive inputs ($x > 0$), the slope is a constant 1.
 - Gradients can flow "uninterrupted" through the network, no matter how deep.
 - This makes training *much* faster and more stable.

Variations: Leaky ReLU & Softmax

Leaky ReLU

- Fixes the "Dying ReLU" problem (where $x < 0$ kills the gradient).
- $\text{Leaky ReLU}(x) = \max(0.01x, x)$
- Allows a small, non-zero gradient when $x < 0$.
- In practice, it doesn't always perform better than ReLU, but is a good alternative to try.

Softmax

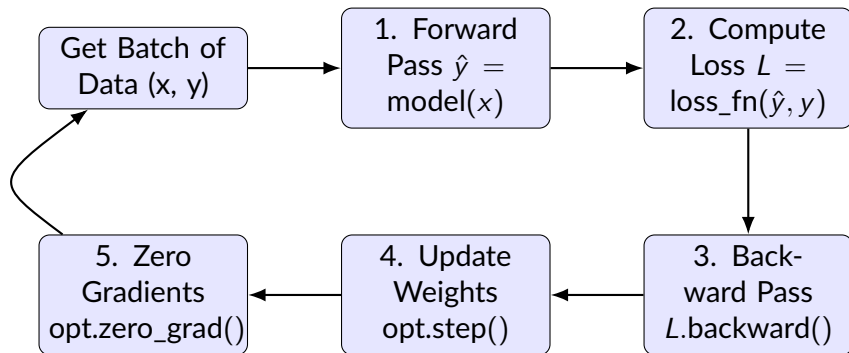
- **Not a hidden layer activation.**
- Used **only for multi-class classification output layers.**
- Converts a vector of scores (logits) into a probability distribution.

$$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

- **Properties:** All outputs are (0, 1) and all outputs sum to 1.

The Training Loop (Big Picture)

How do we find the "right" values for all the weights (w) and biases (b)?



Loss Functions: A Quick Recap

The loss function $L(\hat{y}, y)$ is our "error signal".

For Regression (predicting a number):

- **Mean Squared Error (MSE):** $L = \frac{1}{N} \sum (y - \hat{y})^2$
- Penalizes large errors heavily.
- PyTorch: `nn.MSELoss`

For Classification (predicting a class):

- **Cross-Entropy Loss:** $L = -\frac{1}{N} \sum y \log(\hat{p})$
- Measures the "distance" between two probability distributions (the true labels and our predictions).
- PyTorch: `nn.CrossEntropyLoss`
- **Note:** `nn.CrossEntropyLoss` expects raw **logits** from the model (the scores *before* Softmax). It applies Softmax internally for stability.

Optimization (Gradient Descent)

We can't see the whole loss landscape. We have to find the bottom by "feel".

- 1 Start at a random point (random θ).
- 2 Compute the **gradient** (slope) at that point: $\nabla_{\theta} L$.
- 3 The gradient is a vector that points **uphill** (steepest *ascent*).
- 4 We want to go **downhill**, so we take a small step in the *opposite* direction: $\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L$
- 5 η is the **learning rate**, a small number (e.g., 0.01) that controls our step size.
- 6 Repeat.

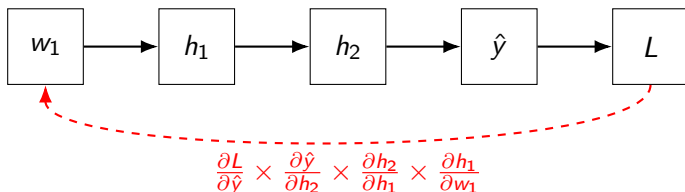
How?

How do we efficiently compute the gradient $\nabla_{\theta} L$ for *millions* of parameters in a deep network? **Backpropagation**.

Backpropagation (Conceptual)

Backpropagation is just the **Chain Rule** from calculus, applied efficiently.

- **Goal:** We need the gradient of the Loss L w.r.t. a weight w_1 .
- **Problem:** w_1 is at the *start* of the network, L is at the *end*.
- **Chain Rule:** $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h_2} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial w_1}$



In PyTorch

You **never** have to do this by hand. You call `loss.backward()`, and PyTorch's **Autograd** system computes all gradients automatically.

Flavors of Gradient Descent

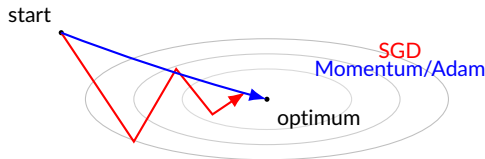
Whose gradient do we compute?

- Batch Gradient Descent
 - Compute gradient using the **entire** dataset.
 - **Pros:** Stable, accurate gradient.
 - **Cons:** Extremely slow. Impossible for large datasets (e.g., ImageNet).
- Stochastic Gradient Descent (SGD)
 - Compute gradient using **one sample** at a time.
 - **Pros:** Very fast updates.
 - **Cons:** Very noisy gradient. Loss will be unstable.
- Mini-Batch Gradient Descent (The Standard)
 - Compute gradient using a small **batch** (e.g., 32 or 64 samples).
 - **This is what everyone calls "SGD" in practice.**
 - **Pros:** Good balance. Fast, stable, parallelizable (via 'DataLoader').

The Problem with Vanilla SGD

The "vanilla" update rule $\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L$ has problems:

- **Slow Convergence:** If the loss landscape is a long, narrow "ravine", the gradient will just oscillate side-to-side instead of moving along the bottom.
- **Local Minima:** It can get "stuck" in a small dip (a local minimum) and think it's at the bottom, when the true global minimum is elsewhere.



SGD oscillates (red) while a better optimizer moves directly (blue).

The Solution

Use an **Optimizer** that adapts the learning process, using not just the current gradient, but also its history.

Optimizer: SGD with Momentum (SGDM)

High-level intuition: Plain SGD reacts immediately to gradient changes and zig-zags. Momentum smooths the direction by keeping part of the previous update, like pushing a heavy object that continues moving despite small bumps.

Formulation

$$g_t = \nabla_{\theta} L_t,$$

$$v_t = \beta v_{t-1} - \eta g_t \quad (\text{blend old velocity with new push}),$$

$$\theta_{t+1} = \theta_t + v_t \quad (\text{take a step following the velocity}).$$

Here $\beta \in [0, 1)$ is the *momentum* (how much past motion you keep) and η is the step size.

```
1 import torch.optim as optim
2 optimizer = optim.SGD(model.parameters(), lr=0.01,
    momentum=0.9)
```

Optimizer: Adagrad

High-level intuition: Different parameters need different learning rates. Adagrad automatically gives larger steps to rarely-updated parameters and smaller steps to frequently-updated ones, which helps with sparse features.

Formulation

$$g_t = \nabla_{\theta} L_t,$$

$$r_t = r_{t-1} + g_t^{\odot 2} \quad (\text{ever-growing counter of each coordinate's behavior}),$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{r_t} + \epsilon} \quad (\text{coordinates with lots of action (large } r_t) \text{ slow down}).$$

Note the denominator keeps increasing over time, which can make steps too tiny late in training.

```
1 import torch.optim as optim
2 optimizer = optim.Adagrad(model.parameters(), lr=0.01,
    eps=1e-10)
```

Optimizer: RMSprop

High-level intuition: Adagrad's steps shrink too fast because it keeps accumulating old gradients. RMSprop fixes this by using a moving average, so it adapts to recent gradient magnitudes and keeps learning at a healthy pace.

Formulation

$$g_t = \nabla_{\theta} L_t,$$

$$s_t = \rho s_{t-1} + (1 - \rho) g_t^{\odot 2} \quad (\text{recent-average of squared slopes}),$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon} \quad (\text{normalize step: small slopes} \Rightarrow \text{larger steps}).$$

$\rho \in [0, 1)$ controls how fast you forget; ϵ prevents division by zero.

```
1 import torch.optim as optim
2 optimizer = optim.RMSprop(model.parameters(), lr=0.001,
    alpha=0.99, eps=1e-8, momentum=0.0)
```

Optimizer: Adam

High-level intuition: Adam blends momentum (smooth direction) and RMS-like scaling (safe step size) into one method, giving stable and fast convergence with minimal tuning.

Formulation

$$g_t = \nabla_{\theta} L_t,$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{direction of travel (momentum)}),$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^{\odot 2} \quad (\text{road roughness (scale)}),$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{fixes small-sample underestimation}),$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad [\text{move along trusted direction with tempered step}].$$

```
1 import torch.optim as optim
2 optimizer = optim.Adam(model.parameters(), lr=1e-3,
    betas=(0.9, 0.999), eps=1e-8)
```

High-level intuition: AdamW separates weight decay from the adaptive update, making regularization work properly and improving generalization compared to standard Adam with L2.

Formulation

$\theta_{t+\frac{1}{2}} = \theta_t - \eta\lambda\theta_t$ (decoupled weight decay: a separate small pull toward zero)

$\theta_{t+1} = \theta_{t+\frac{1}{2}} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ (regular Adam step on the decayed weights).

Here $(\hat{m}_t, \hat{v}_t)$ are the Adam bias-corrected estimates; λ is the decay strength.

```
1 import torch.optim as optim
2 optimizer = optim.AdamW(model.parameters(), lr=1e-3,
    betas=(0.9, 0.999), eps=1e-8, weight_decay=0.01)
```

How to Choose Your Optimizer

- **Default Choice (90% of cases): AdamW**

- **Why:** It's fast, converges reliably with minimal tuning, and correctly implements weight decay (L2 regularization), which helps prevent overfitting.
- Start with $lr=1e-3$, $weight_decay=1e-2$.

- **For Best Performance (SOTA): SGD + Momentum**

- **Why:** If you are willing to carefully tune the learning rate, momentum, and learning rate schedule, SGD+Momentum can often achieve slightly better final validation accuracy than Adam.
- Requires much more manual effort.

- **Historical / Niche Use:**

- Adagrad, RMSprop were important steps toward Adam. They are less commonly used today.

Initialization: The "Why"

Problem: How do we set the *initial* values for weights w ?

- **What if we set all weights to 0?**

- **Bad!** Every neuron in a layer will be identical.
- They will all get the same input, compute the same output, and receive the same gradient.
- The network will never learn. This is "symmetry breaking". We must **break symmetry** with random values.

- **What if we use large random numbers?**

- **Bad!** The inputs to Sigmoid/Tanh will be very large.
- The activations will saturate (be near -1 or 1).
- The gradients will be ≈ 0 (Vanishing Gradients). Training stalls.

Initialization: The "How" (Xavier & He)

The Goal: We need to initialize weights so that the *variance* (spread) of activations remains roughly the same as we go deeper.

Xavier / Glorot Initialization

- For **Tanh** and **Sigmoid** activations.
- Samples from a distribution with variance $\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$.
- In PyTorch: `nn.init.xavier_uniform_(tensor)`

He / Kaiming Initialization

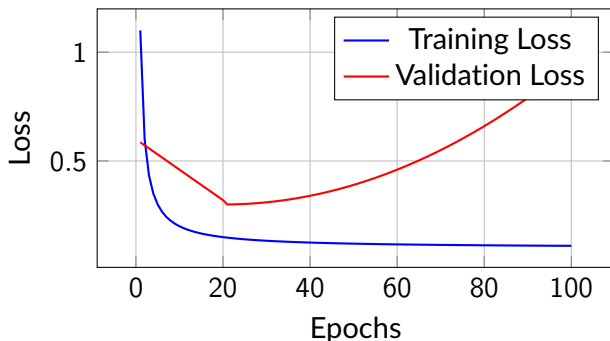
- For **ReLU** activations (and its variants).
- ReLU "kills" half the activations, so it needs a different variance.
- Samples from a distribution with variance $\text{Var}(w) = \frac{2}{n_{\text{in}}}$.
- In PyTorch: `nn.init.kaiming_normal_(tensor)`

In Practice

PyTorch's `nn.Linear` and `nn.Conv2d` layers already use Kaiming He initialization by default! You usually don't have to do this manually.

The Problem: Overfitting

The Symptom: Your model trains perfectly (low training loss) but fails on new data (high test loss).



The Diagnosis: The model is too complex. It has "memorized" the training data (including its noise) instead of learning the general pattern.

The Cure: Regularization

What is Regularization?

Regularization = Any technique that constrains the model to make it simpler.

- **Idea:** We are fighting for **generality**.
- We want to force the model to find the *simplest possible solution* that fits the data.
- This is based on "Occam's Razor": the simplest explanation is usually the best.
- A simpler model is less likely to "memorize" noise and more likely to capture the true, general pattern.

Common Techniques:

- L1 / L2 Regularization (Weight Decay)
- Dropout
- Data Augmentation
- Early Stopping
- Label Smoothing

Regularization (1): L1 & L2 (Weight Decay)

The Idea: "Simple" = "small weights". A model with huge, extreme weights is a sign of overfitting.

How it works: We add a **penalty term** to the loss function.

- L2 Regularization (Weight Decay):

$$L_{\text{new}} = L_{\text{original}} + \lambda \sum w^2$$

- **Effect:** Encourages all weights to be small and distributed.
- This is the **most common** type of regularization.
- In PyTorch: Set `weight_decay=0.01` in the optimizer.

- L1 Regularization (Lasso):

$$L_{\text{new}} = L_{\text{original}} + \lambda \sum |w|$$

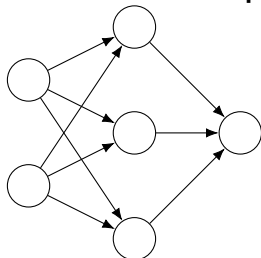
- **Effect:** Encourages *sparsity* (pushes many weights to be exactly 0).

Regularization (2): Dropout

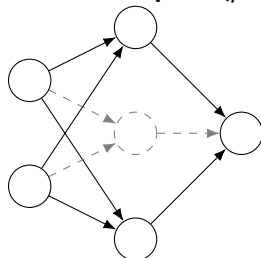
The Idea: The model is "cheating" by having neurons co-adapt (e.g., "I'll only work if Neuron 27 is active").

How it works: During training, for each forward pass, randomly "drop" (set to 0) a fraction p (e.g., $p = 0.5$) of neurons.

Full Network w/o Dropout



Network w/ Dropout ($p = 0.5$)



- **Effect:** Prevents "co-adaptation". Neurons are forced to learn robust, independent features.
- **IMPORTANT:** Dropout is **only** active during training (`model.train()`). It is **disabled** during evaluation (`model.eval()`).

Dropout in PyTorch: nn.Dropout

```
1 class MLPWithDropout(nn.Module):
2     def __init__(self, input_dim, hidden_dim, output_dim,
3         p_drop=0.5):
4         super().__init__()
5         self.fc1 = nn.Linear(input_dim, hidden_dim)
6         self.fc2 = nn.Linear(hidden_dim, hidden_dim)
7         self.fc3 = nn.Linear(hidden_dim, output_dim)
8         # Initialize the dropout layer
9         self.dropout = nn.Dropout(p=p_drop)
10    def forward(self, x):
11        x = self.fc1(x)
12        x = F.relu(x)
13        x = self.dropout(x) # Apply dropout after activation
14        x = self.fc2(x)
15        x = F.relu(x)
16        x = self.dropout(x) # Apply dropout again
17        x = self.fc3(x)
18        return x
```

Note: Don't forget to call `model.eval()` before testing!

Regularization (3): Data Augmentation

The Idea: If you're overfitting, you don't have enough data. So... **get more data!** If you can't collect more, *create* more by "augmenting" your existing data.

For Images:

- **Random Flip:** Horizontally flip the image.
- **Random Crop:** Take a random small crop from a large image.
- **Random Rotation:** Rotate the image by a few degrees.
- **Color Jitter:** Randomly change brightness, contrast, saturation.

For Text:

- **Back-translation:** Translate (en $\rightarrow$ fr $\rightarrow$ en) to get a paraphrased sentence.
- **Synonym Replacement:** Randomly swap words with synonyms.

This teaches the model **invariance** (e.g., a "cat" is still a "cat" if it's flipped or in different lighting).

Data Augmentation in PyTorch

For Images (using torchvision.transforms):

```
1 import torchvision.transforms as transforms
2 from torchvision import datasets
3 import torch
4 # Define augmentation pipeline
5 train_transform = transforms.Compose([
6     transforms.RandomHorizontalFlip(p=0.5),      # Random flip
7     transforms.RandomRotation(degrees=15),      # Random rotation
8     transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation
9                             =0.2),
10    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
11    transforms.ToTensor(),
12    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
13    0.224, 0.225])
14 ])
15 # Apply to dataset
16 train_dataset = datasets.ImageFolder(root='path/to/train/data',
17                                     transform=train_transform)
```

Regularization (4): Early Stopping

The Idea: If overfitting happens *over time* (as the red line shows), let's just stop training before it gets bad!

How it works:

- 1 Monitor your **validation loss** (loss on data the model hasn't trained on) at the end of every epoch.
- 2 Keep a copy of the model weights that have the **best validation loss so far**.
- 3 If the validation loss doesn't improve for N epochs (e.g., $N = 5$, the "patience"), **stop training**.
- 4 Use the "best" weights you saved, not the final ones.

Early Stopping in PyTorch

```
1 # Early stopping in PyTorch
2 best_loss = float('inf')
3 patience_counter = 0
4 for epoch in range(epochs):
5     # ... training code ...
6     val_loss = validate_model()
7     if val_loss < best_loss:
8         best_loss = val_loss
9         patience_counter = 0
10        torch.save(model.state_dict(), 'best_model.pth')
11    else:
12        patience_counter += 1
13        if patience_counter >= 5:
14            model.load_state_dict(torch.load('best_model
15                                     .pth'))
16            break
```

Regularization (5): Label Smoothing

The Idea: Overfitting can happen when the model becomes "over-confident".

- For a "cat" image, the label is $[1, 0, 0]$ (cat, dog, fish). This is a "hard" target. The model is trained to output $\hat{p} \rightarrow [1.0, 0.0, 0.0]$.
- To get $\hat{p}$ to 1.0, the logits (inputs to softmax) have to go to $+\infty$. This encourages extreme weights.

Label Smoothing: We "smooth" the target labels slightly.

- Instead of $[1, 0, 0]$, we use a "soft" target like:

$$[1 - \epsilon, \epsilon/2, \epsilon/2], \quad \text{e.g., } [0.9, 0.05, 0.05] \text{ for } \epsilon = 0.1$$

- Prevents over-confidence and acts as a regularizer.

In PyTorch

Since PyTorch 1.10, `nn.CrossEntropyLoss` has a `label_smoothing` argument: `loss_fn = nn.CrossEntropyLoss(label_smoothing=0.1)`

Normalization: The Problem

Problem: Internal Covariate Shift

- As the network trains, the weights in *Layer 1* change.
- This changes the *distribution* (mean μ , variance σ^2) of activations that *Layer 2* receives.
- From Layer 2's perspective, its input data is *constantly shifting*.
- This forces Layer 2 to constantly re-adapt, slowing down training. It's like trying to hit a moving target.

The Solution

Normalize the activations inside the network to keep their mean and variance stable during training.

Normalization (1): Batch Normalization (BatchNorm)

The Idea: Normalize the activations **within a mini-batch**.

For a mini-batch $\mathcal{B}$ with m samples:

- 1 **Calculate Batch Stats:** Find the mean ($\mu_{\mathcal{B}}$) and variance ($\sigma_{\mathcal{B}}^2$) of the activations for each feature/channel **across the batch**.

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$$

- 2 **Normalize:**

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

- 3 **Scale and Shift:** This normalization is too restrictive (forces mean=0, var=1). We let the network *learn* the best mean/variance.

$$y_i = \gamma \hat{x}_i + \beta$$

where γ and β are new **learnable parameters**.

Location: Applied *after* the linear layer and *before* the activation.

BatchNorm in PyTorch: nn.BatchNorm2d

```
1 class ConvBlock(nn.Module):
2     def __init__(self, in_channels, out_channels):
3         super().__init__()
4         self.conv = nn.Conv2d(in_channels, out_channels,
5                                kernel_size=3, padding=1)
6
7         # BatchNorm layer has learnable parameters (
8             gamma, beta),
9         # so it must be initialized in __init__
10        # We specify the number of *channels* (features)
11        self.bn = nn.BatchNorm2d(num_features=
12            out_channels)
13
14    def forward(self, x):
15        x = self.conv(x)
16        x = self.bn(x)    # Apply BN
17        x = F.relu(x)     # Apply ReLU
18        return x
```

Normalization (2): Layer Normalization (LayerNorm)

Problem with BatchNorm: It depends on the batch size.

- If batch size is 1, variance is undefined.
- If batch sizes are small, the μ_B and σ_B^2 are "noisy" estimates.
- This makes it unsuitable for **RNNs** or **Transformers** where sequence lengths vary (and batch size is often 1).

Layer Normalization (LayerNorm)

- **Idea:** Normalize **within a single sample**, across its features.
- **Stats:** Calculate μ and σ^2 **across all features (the layer)** for one sample x_i .
- It is **independent of the batch size**.

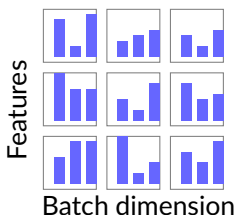
LayerNorm in PyTorch: nn.LayerNorm

```
1 class TransformerBlock(nn.Module):
2     def __init__(self, hidden_size):
3         super().__init__()
4         self.attention = nn.MultiheadAttention(hidden_size, num_heads
5         =8)
6         self.ln1 = nn.LayerNorm(hidden_size)
7         self.ln2 = nn.LayerNorm(hidden_size)
8         self.ffn = nn.Sequential(
9             nn.Linear(hidden_size, 4 * hidden_size),
10            nn.ReLU(),
11            nn.Linear(4 * hidden_size, hidden_size)
12        )
13    def forward(self, x):
14        # x shape: [seq_len, batch_size, hidden_size]
15        attn_out, _ = self.attention(x, x, x)
16        x = x + attn_out # Residual connection
17        x = self.ln1(x) # LayerNorm after attention
18
19        ffn_out = self.ffn(x)
20        x = x + ffn_out # Residual connection
21        x = self.ln2(x) # LayerNorm after FFN
22        return x
```

BatchNorm vs. LayerNorm (Visualized)

Batch Normalization (for CNNs)

- Normalizes across the *batch* dimension.
- "For this one channel, what is the avg/stddev across all samples in the batch?"

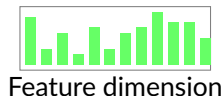


Compute mean/std across batch

Layer Normalization (for RNNs/Transformers)

- Normalizes across the *feature* dimension.
- "For this one sample, what is the avg/stddev across all its features?"

Compute mean/std across features



How to Choose Your Normalization Layer

- **For CNNs (Images, Grids): Use BatchNorm**

- `nn.BatchNorm1d` for 1D data (MLPs)
- `nn.BatchNorm2d` for 2D data (Images)
- **Why:** It normalizes **across the batch**. It assumes that all samples in the batch are i.i.d. and that the batch size is large enough (e.g., > 16) to get good statistics.

- **For RNNs/Transformers (Sequences): Use LayerNorm**

- `nn.LayerNorm`
- **Why:** It normalizes **across the features** of a **single sample**. This is essential for sequential data where:
 - Sequence lengths vary.
 - Batch sizes can be very small (even 1), which would make batch statistics unstable.

Stability: Gradient Clipping

Problem: In deep networks (especially RNNs), gradients can become massive, leading to $\nabla_{\theta} L \rightarrow \infty$.

- This "exploding gradient" causes the update $\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L$ to be huge.
- The parameters "explode" to NaN (Not a Number).
- Training fails instantly.

Solution: Gradient Clipping

- **Idea:** If the gradient is too big, "clip" it to a maximum value.
- After `loss.backward()`, *before* `optimizer.step()`, we add one line:

```
1 # --- Inside training loop ---
2 loss = ...
3 loss.backward()
4 # Clip the gradients (rescales all grads if total norm >
   1.0)
5 torch.nn.utils.clip_grad_norm_(model.parameters(),
   max_norm=1.0)
6 optimizer.step()
```

Your Mission: Build a robust classifier for Fashion-MNIST using all the deep learning basics techniques we've covered.

- **Step 1: Setup and Data Loading**

- Load the Fashion-MNIST dataset using `torchvision.datasets.FashionMNIST`
- Split into training (55k), validation (5k), and test (10k) sets
- Apply basic normalization to the images (mean=0.5, std=0.5)

- **Step 2: Model Architecture**

- Create an MLP with **at least 3 hidden layers** using `nn.Module`
- Apply **ReLU activations** after each hidden layer
- Include **BatchNorm1d** after each linear layer (before activation)
- Add **Dropout** ($p=0.3$) after the first two hidden layers

- **Step 3: Training Configuration**

- Use `nn.CrossEntropyLoss` as your loss function
- Implement **AdamW optimizer** with learning rate 0.001 and weight decay 0.01
- Apply **gradient clipping** (`max_norm=1.0`) during training
- Implement **early stopping** with `patience=5` based on validation loss

Convolutional Networks

Problem with MLPs for Images

Question: Why can't we just use a big MLP on an image?

Let's try on a "small" 256x256 RGB image:

- Input dimensions: $256 \times 256 \times 3 = 196,608$ features.
- If our first hidden layer has 1,000 neurons (modest)...
- Number of weights = $196,608 \times 1,000 = \mathbf{196.6 \text{ Million}}$
- This is for *one layer*. The model would be **enormous**, slow, and would overfit instantly.

Two Core Problems with MLPs:

- 1 **Not parameter-efficient:** MLPs are "fully-connected". They don't assume any structure.
- 2 **Don't respect image structure:**
 - **Local patterns:** Pixels near each other are related (e.g., an "eye"). An MLP treats the top-left pixel and bottom-right as equally related.
 - **Translation Invariance:** A "cat" in the top-left corner is the same "cat" as one in the bottom-right. An MLP would have to re-learn "cat" at *every possible position*.

Convolutional Neural Networks (CNNs) solve MLP problems with three key ideas:

- **Convolution Operation**

- **Idea:** Patterns are **local**.
- **Solution:** Small filters slide over image as **local feature detectors**.

- **Parameter Sharing**

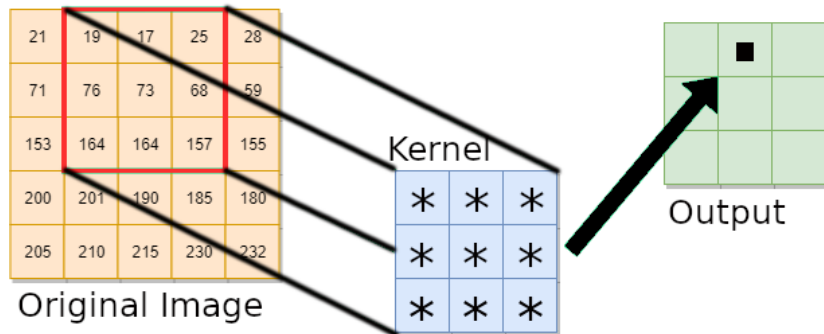
- **Idea:** Features are **translation-invariant**.
- **Solution:** Same filter weights used everywhere.
- **Benefit:** 3x3 filter needs only 9 weights vs. 196M for MLP.

- **Pooling**

- Downsamples feature maps for efficiency and shift invariance.

The Convolution Operation (Visual)

A "filter" or kernel (e.g., 3x3) slides over the input image. At each step, it computes an element-wise multiplication and sums the result (a dot product).



The Convolution Operation (Math)

The output of one "neuron" in a feature map is:

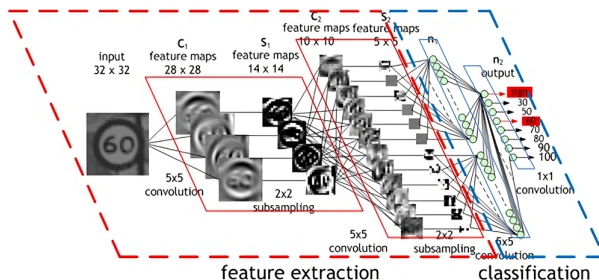
$$\text{Output} = \text{Activation} \left(\sum_{i=1}^K \sum_{j=1}^K \text{Filter}_{i,j} \times \text{InputPatch}_{i,j} + b \right)$$

- The operation represents a dot product between the filter and the input patch, which is the core of the convolution operation.
- The **Filter** (e.g., 3×3) contains the weights W .
- The **Bias** b is a single number, added to the result of the sum.
- **Parameter Sharing:** The *same* W and b are used for every single position as the filter slides.
- The output (e.g., 3×3 in the previous slide) is called a **Feature Map**.

CNN Filters as Feature Detectors

The network **learns** the values of these filters during training. Different filters learn to detect different patterns.

- **Early Layers** (e.g., Conv 1) learn simple features, e.g., edges (horizontal, vertical, diagonal), corners, color blobs
- **Deeper Layers** (e.g., Conv 3) combine these simple features to learn complex patterns, e.g., textures (stripes, spots, mesh), simple objects (eyes, noses, wheels)
- **Final Layers** (e.g., Conv 5) combine those patterns to learn full objects, e.g., faces, cars, dogs, buildings



Channels

A real CNN uses multiple channels.

- **Input Channels (C_{in}):** A color image (RGB) has 3 input channels.
- **Output Channels (C_{out}):** A convolution layer applies multiple filters (e.g., 64 filters) to the input.
- Each filter produces **one** output channel (one feature map).
- The final output is a **stack** of feature maps.

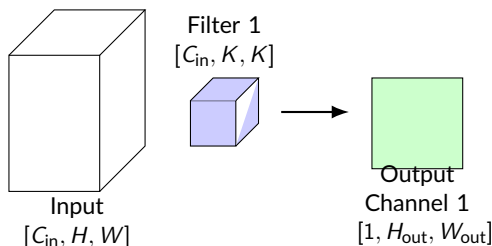
A single nn.Conv2d layer:

- **Input Shape:** [Batch, C_{in} , H_{in} , W_{in}]
- **Filter Shape:** [C_{out} , C_{in} , K_H , K_W]
- **Output Shape:** [Batch, C_{out} , H_{out} , W_{out}]

```
1 # 3 input channels (RGB)
2 # 64 output channels (64 filters)
3 # 3x3 kernel size
4 conv1 = nn.Conv2d(in_channels=3, out_channels=64,
                    kernel_size=3)
```

Convolution in Detail (Channels)

- An input with C_{in} channels (e.g., 3 for RGB) is not 2D, it's 3D.
- Therefore, the filter must also be 3D. It must have the **same depth** as the input.
- A "3x3 filter" on an RGB image is actually a **3x3x3** block of weights.
- This 3D filter slides over the 3D input, computes *one* dot product ($3 \times 3 \times 3 = 27$ multiplications), and produces *one* number.
- To get C_{out} (e.g., 64) output channels, we need *64 different* 3x3x3 filters.

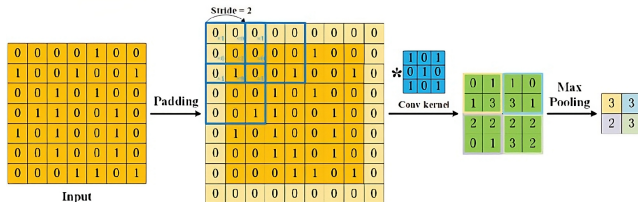


Stride and Padding

We have two controls for the convolution:

- Stride**
- How many pixels to "step" the filter at a time.
 - Default stride=1 (moves 1 pixel at a time).
 - stride=2 will skip every other pixel, **downsampling** the image (output will be half the size).

- Padding**
- Convolutions "shrink" the image (a 5x5 input with 3x3 filter gives a 3x3 output).
 - We **pad** the input with zeros around the border to preserve its size.
 - **Rule of thumb:** For a $K \times K$ filter, use padding = $(K-1)/2$ to keep the size the same.



Calculating Output Size

This is a critical formula for designing CNNs.

Given:

- W_{in} = Input Width
- K = Kernel (Filter) Size
- P = Padding
- S = Stride

The output width W_{out} is:

$$W_{out} = \frac{W_{in} - K + 2P}{S} + 1$$

(The same formula applies to the height H).

Example: Input is 32×32 . Filter is 3×3 ($K = 3$). Stride is 1 ($S = 1$).

- **No Padding** ($P = 0$): $W_{out} = (32 - 3 + 0)/1 + 1 = 30$. *Output is 30×30 (it shrinks).*
- **With "Same" Padding** ($P = 1$): $W_{out} = (32 - 3 + 2)/1 + 1 = 32$. *Output is 32×32 (it stays the same).*

Conv2D in PyTorch: nn.Conv2d

```
1 import torch
2 import torch.nn as nn
3 # Input tensor: [batch, channels, height, width]
4 x = torch.randn(1, 3, 64, 64) # 1 image, 3 channels, 64
    x64 pixels
5 # Conv2d with stride and padding
6 conv2d = nn.Conv2d(
7     in_channels=3,          # RGB input
8     out_channels=32,        # 32 filters
9     kernel_size=3,         # 3x3 filters
10    stride=2,               # Stride: move 2 pixels at a
        time
11    padding=1               # Padding: pad height and width
        dimensions
12 )
13 output = conv2d(x)
14 print(f"Input shape: {x.shape}")      # [1, 3, 64, 64]
15 print(f"Output shape: {output.shape}") # [1, 32, 32, 32]
```

Pooling Layers

Idea: Aggressively downsample the feature map.

- Max Pooling (`nn.MaxPool2d`)
 - Takes a window (e.g., 2×2) and outputs the **maximum** value from that window.
 - **Effect:** Selects the *strongest* feature signal. "Was there an edge in this 2×2 region?"

```
1 import torch.nn as nn
2 # Max Pooling
3 max_pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
```

- Average Pooling (`nn.AvgPool2d`)
 - Takes a 2×2 window and outputs the **average** value.
 - **Effect:** Blurs/smooths the features. Less common, but used in ResNet heads.

```
1 import torch.nn as nn
2 # Average Pooling
3 avg_pool = nn.AvgPool2d(kernel_size=2, stride=2, padding=0)
```

Pooling has **no learnable parameters**. It is a fixed operation.

Why Pooling?

- **1. Computational Efficiency:**

- A 2×2 pooling layer (with stride 2) cuts the feature map size by 75% (e.g., $32 \times 32 \rightarrow 16 \times 16$).
- This drastically reduces the number of parameters and computations needed in the *next* convolutional layer.

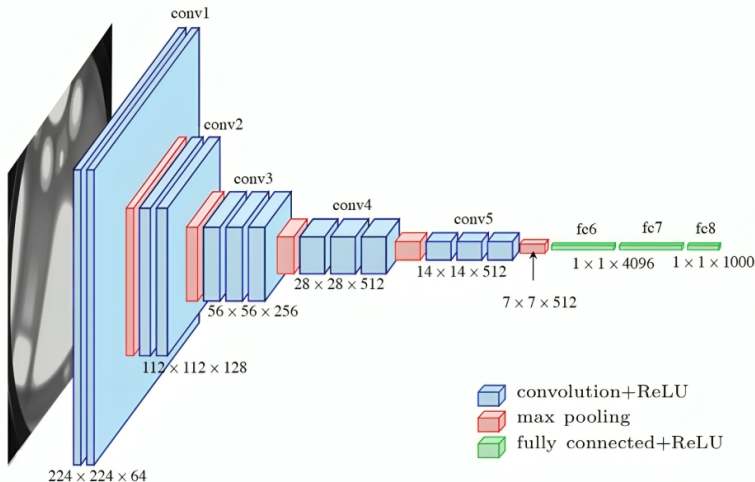
- **2. Local Translation Invariance:**

- By taking the 'max' over a 2×2 region, the network becomes "less sensitive" to the exact position of the feature.
- An edge in position (4, 4) and an edge in (4, 5) might both result in the same 'max' output.
- This helps the model generalize better.

A Full CNN Architecture (e.g., VGG-style)

A typical CNN architecture is a repeating pattern:

(CONV → ReLU → POOL) × N → (FC → ReLU) × M → FC (Output)



A Simple CNN in PyTorch

```
1 class SimpleCNN(nn.Module):
2     def __init__(self, num_classes=10):
3         super().__init__()
4         # Convolutional Base
5         self.conv_base = nn.Sequential(
6             # Input: 3 x 32 x 32
7             nn.Conv2d(3, 32, kernel_size=3, padding=1),
8             nn.ReLU(),
9             nn.MaxPool2d(kernel_size=2, stride=2), # -> 32 x 16 x 16
10            nn.Conv2d(32, 64, kernel_size=3, padding=1),
11            nn.ReLU(),
12            nn.MaxPool2d(kernel_size=2, stride=2) # -> 64 x 8 x 8
13        )
14        # Classifier Head
15        self.classifier = nn.Sequential(
16            nn.Flatten(), # -> 64 * 8 * 8 = 4096
17            nn.Linear(64 * 8 * 8, 512),
18            nn.ReLU(),
19            nn.Linear(512, num_classes)
20        )
21    def forward(self, x):
22        x = self.conv_base(x)
23        x = self.classifier(x)
24        return x
```

The Problem of Deep Networks

Question: We know stacking layers builds better features. Why not stack 100 layers? Or 1000?

Problem 1: Vanishing Gradients

- In backpropagation, we multiply gradients:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a_N} \times \frac{\partial a_N}{\partial a_{N-1}} \times \cdots \times \frac{\partial a_2}{\partial w_1}.$$

- If each gradient term is small (< 1), their product becomes tiny:
 $0.9 \times 0.9 \times \cdots \times 0.9 \rightarrow 0.$
- The gradients for the early layers "vanish", and they stop learning. (ReLU helps, but doesn't fix it entirely).

Problem 2: Degradation

- Even with good initialization and BatchNorm, a 56-layer network often performs *worse* than a 20-layer network.
- The network has trouble learning a simple *identity mapping* (i.e., just passing the input through $H(x) = x$).

Residual Connections (ResNet)

The Idea: Make it trivial for the network to learn an identity mapping.

Plain Block

$$x \rightarrow \boxed{\text{Conv Block}} \rightarrow H(x)$$

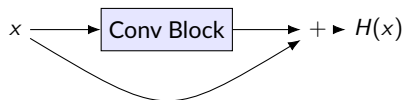
The block must learn the entire transformation $H(x)$.



Residual Block

$$H(x) = F(x) + x$$

The block only learns the *residual* $F(x)$.



- The "skip connection" adds the original input x to the output of the block.
- If the best solution is an identity map ($H(x) = x$), the network can **easily learn** to set the weights in $F(x)$ to zero.
- This "residual" formulation is much easier to optimize.
- It also allows gradients to flow directly back through the skip connection, **solving the vanishing gradient problem**.

ResNet Block in PyTorch

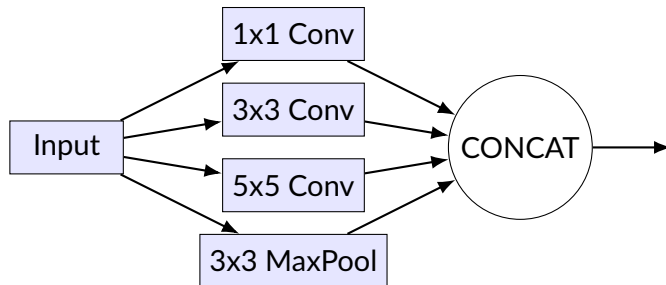
```
1 class ResidualBlock(nn.Module):
2     def __init__(self, channels):
3         super().__init__()
4         self.conv1 = nn.Conv2d(channels, channels, kernel_size=3,
5                                 padding=1)
6         self.bn1 = nn.BatchNorm2d(channels)
7         self.conv2 = nn.Conv2d(channels, channels, kernel_size=3,
8                                 padding=1)
9         self.bn2 = nn.BatchNorm2d(channels)
10
11     def forward(self, x):
12         # Store the original input (the residual)
13         residual = x
14         # Pass through the block
15         out = F.relu(self.bn1(self.conv1(x)))
16         out = self.bn2(self.conv2(out))
17         # Add the residual back
18         out = out + residual
19         # Apply final activation
20         out = F.relu(out)
21         return out
```

Common Blocks: Inception Module

The Problem: What kernel size is best? 3×3 ? 5×5 ? 1×1 ?

The Idea (GoogLeNet / Inception): Why not all of them?

- An "Inception module" runs multiple convolutional branches in **parallel** and **concatenates** their outputs.
- **1×1 Conv:** A cheap way to reduce the number of channels (a "bottleneck") before the expensive 3×3 and 5×5 convs.
- This allows the network to be "wider" instead of just "deeper".



Common Blocks: Depthwise Separable Conv

The Problem: A standard 3×3 convolution on a 64-channel input is expensive. It requires *64 different* $3 \times 3 \times 64$ filters.

The Idea (MobileNet): Split the convolution into two steps.

- 1. Depthwise Convolution
 - Apply one 3×3 filter to *each* input channel independently.
 - This captures spatial patterns (like "edge") but doesn't mix channel information.
 - (Input: $C \times H \times W$) $\rightarrow$ (Output: $C \times H \times W$)
- 2. Pointwise Convolution (1x1 Conv)
 - Apply a 1×1 convolution to mix the channel information.
 - This is just a linear layer applied at every pixel.
 - (Input: $C \times H \times W$) $\rightarrow$ (Output: $C_{\text{out}} \times H \times W$)

This is *dramatically* more parameter-efficient than a standard convolution and is the foundation of modern, fast mobile-friendly networks (e.g., MobileNet).

Practice (To Be Submitted)

Your Mission: Build a CNN classifier for CIFAR-10 using modern CNN building blocks.

- **Step 1: Setup and Data Loading**

- Load the CIFAR-10 dataset using `torchvision.datasets.CIFAR10`
- Use the standard train/test split (50k/10k samples)
- Apply normalization transforms (mean=[0.4914, 0.4822, 0.4465], std=[0.2023, 0.1994, 0.2010])

- **Step 2: Model Architecture**

- Create a CNN with at least 2 different types of blocks: **ResNet blocks** and **Inception modules**
- Include **BatchNorm2d** after each convolution layer
- Use **Dropout** (p=0.3) in the classifier head
- Implement **Global Average Pooling** instead of Flatten+Linear for the final layers

- **Step 3: Training Configuration**

- Use `nn.CrossEntropyLoss` as your loss function
- Implement **Adam optimizer** with learning rate 0.001
- Train for at least 20 epochs with **learning rate scheduling** (reduce by half every 5 epochs) and then evaluate on test split

Sequence Modeling

Problem with CNNs/MLPs for Sequences

What is Sequence Data?

- Data where **order matters**.
- **Text**: "man bites dog" vs. "dog bites man"
- **Time Series**: Stock prices, weather data
- **Audio**: Waveforms

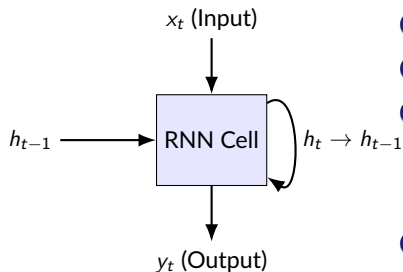
Why can't CNNs or MLPs handle sequences?

- 1 **No sense of "time" or order**: They treat every input as independent. A shuffled sentence would look the same to an MLP.
- 2 **Fixed-size input**: An MLP trained on 10-word sentences can't handle a 20-word sentence. A CNN has a fixed input size.
- 3 **No "memory"**: When processing the 10th word, they have no memory of the first 9 words.

The Solution: Recurrent Neural Networks (RNNs)

The Idea: A network with a "loop" that maintains a "memory" or "state".

The RNN Cell



(Folded)

At each time step t , the RNN cell:

- 1 Takes the **current input** x_t .
- 2 Takes the **previous hidden state** h_{t-1} .
- 3 Computes the **new hidden state** h_t :

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- 4 (Optionally) computes an output y_t :

$$y_t = W_{hy}h_t + b_y$$

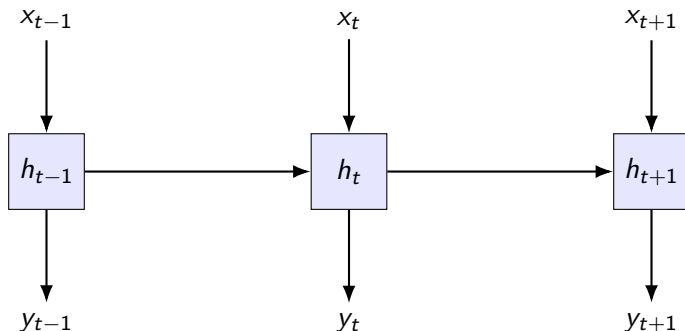
Key Idea

The hidden state h_t is the network's "memory". It's a summary of everything it has seen up to time t .

Unrolling the RNN

It's easier to think of the RNN as a deep network "unrolled" through time.

- This makes it clear: an RNN is just a very deep network where each layer ($t, t + 1, \dots$) **shares the same weights** (W_{hh}, W_{xh}).



Training is done with **Backpropagation Through Time (BPTT)**, which is just standard backpropagation on the unrolled graph.

RNN in PyTorch: nn.RNN

```
1  # Hyperparameters
2  input_size = 10    # e.g., dimension of word embedding
3  hidden_size = 20   # dimension of our "memory"
4  num_layers = 2     # number of stacked RNN layers
5  batch_size = 5
6  seq_len = 7        # length of our sentences
7  # Initialize the RNN layer
8  rnn = nn.RNN(input_size=input_size,
9               hidden_size=hidden_size,
10              num_layers=num_layers,
11              batch_first=True) # Makes shape [Batch, Seq, Feat]
12 # Create dummy input data
13 # Shape: [batch_size, sequence_length, input_size]
14 dummy_input = torch.rand(batch_size, seq_len, input_size)
15 # Pass it through the RNN
16 # output: hidden state for *every* time step
17 # h_n: hidden state for the *last* time step
18 output, h_n = rnn(dummy_input)
19 print("Input shape:", dummy_input.shape)
20 print("Output shape:", output.shape)
21 print("Last hidden shape:", h_n.shape)
```

Gated RNNs

The Problem: The RNN's "memory" h_t is bad. It has to be used for two jobs at once:

- 1 Be the output for the current time step (for prediction).
- 2 Be the "memory" for all future time steps.

This creates a bottleneck.

The Solution: Gated RNNs

- **Idea:** Create a separate, protected "memory" (the **Cell State**, c_t) that is separate from the output (h_t).
- Use "gates" (multiplicative σ functions) to control the flow of information into and out of this protected memory.

This leads to:

- 1 **LSTM** (Long Short-Term Memory)
- 2 **GRU** (Gated Recurrent Unit)

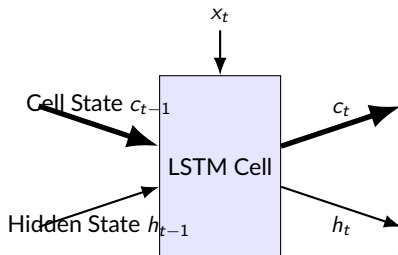
Long Short-Term Memory (LSTM)

An LSTM cell is a more complex RNN cell with **two** states:

h_t (Hidden State) The "output" state. Used for predictions at time t .

c_t (Cell State) The "memory" state. A "conveyor belt" for information.

The cell state c_t flows through the whole chain. The network uses "gates" to read from it, erase from it, and write to it.

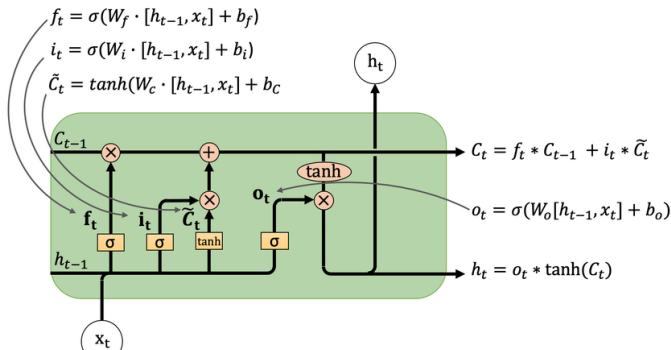


Because c_t is mostly additive ($c_t = c_{t-1} + \dots$), gradients can flow "uninterrupted" back through time, **solving the vanishing gradient problem**.

LSTM Gates

An LSTM controls the cell state c_t using three gates:

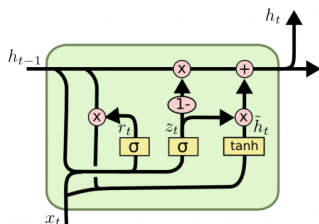
1. **Forget Gate (f_t)** : "How much of the old memory c_{t-1} should we **forget**?"
2. **Input Gate (i_t)** : "How much of the **new information** $\tilde{c}_t$ should we **add**?"
3. **Output Gate (o_t)** : "What part of the (updated) cell state c_t should we **output** as the new hidden state h_t ?"



Gated Recurrent Unit (GRU)

A GRU is a simpler, more modern alternative to an LSTM.

- It **works just as well** as LSTM on many tasks, but is faster to train (fewer parameters).
- It **merges** the hidden state and cell state into **one hidden state** h_t .
- It only has **two gates**:
 - Update Gate (z_t): Combines the "forget" and "input" gates. Decides how much to *keep* from h_{t-1} vs. how much to *add* from the new candidate $\tilde{h}_t$.
 - Reset Gate (r_t): Decides how much of the *previous* memory h_{t-1} to use when calculating the new candidate $\tilde{h}_t$.



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

LSTMs/GRUs in PyTorch

```
1 # Initialize an LSTM layer
2 # API is identical to nn.RNN
3 lstm = nn.LSTM(input_size=input_size,
4                 hidden_size=hidden_size,
5                 num_layers=num_layers,
6                 batch_first=True) # Very important!
7
8 # Initialize a GRU layer
9 gru = nn.GRU(input_size=input_size,
10              hidden_size=hidden_size,
11              num_layers=num_layers,
12              batch_first=True)
13
14 # --- Forward pass ---
15 # LSTM returns (output, (h_n, c_n))
16 # h_n = final hidden state, c_n = final cell state
17 output, (h_n, c_n) = lstm(dummy_input)
18
19 # GRU returns (output, h_n)
20 output, h_n = gru(dummy_input)
```

Advanced RNNs: Bidirectional & Deep

Bidirectional RNN

- "The quick brown fox jumped over the lazy dog."
- To understand "over", it helps to see both "jumped" (past) and "lazy dog" (future).
- A Bidirectional RNN runs **two** RNNs:
 - One (forward) from left-to-right.
 - One (backward) from right-to-left.
- The final output is the **concatenation** of both states.
- `nn.LSTM(..., bidirectional=True)`

Deep (Stacked) RNN

- We can "stack" RNN layers just like we stack CNN/MLP layers.
- The output sequence from Layer 1 becomes the *input* sequence for Layer 2.
- This allows the network to learn hierarchical features *in time*.
- `nn.LSTM(..., num_layers=3)`

The "Seq-to-Seq" Problem

- **Task:** Machine Translation.
- **Input:** "How are you?" (3 words)
- **Output:** "Comment allez-vous?" (3 words)

Problem:

- The "Many-to-Many" RNN we saw earlier (BPTT) requires the input and output length to be identical.
- **Input:** "What is your name?" (4 words)
- **Output:** "Comment t'appelles-tu?" (3 words)
- The lengths don't match! We need a new architecture.

Solution: The Encoder-Decoder Model

The Encoder-Decoder Architecture

We use **two** RNNs to solve this.

- 1. The Encoder (Reader): A "Many-to-One" RNN.
 - It reads the entire input sentence ("What is your name?").
 - It *ignores* all outputs, and just keeps the **final hidden state** h_N .
 - This h_N is the "context vector" or "thought vector" – a summary of the entire input sentence.
- 2. The Decoder (Writer): A "One-to-Many" RNN.
 - Its *initial hidden state* is set to the Encoder's final h_N .
 - It then generates the output sentence word-by-word, conditioned on that initial "thought".

This model decouples input and output length. It reads the whole input, "thinks", then writes the whole output. This was the state-of-the-art until... Transformers!

Recap: Deep Learning Fundamentals

1. Deep Learning Basics

- Building blocks: Neurons, layers, activations (ReLU)
- Training: Loss functions, backpropagation, optimizers (Adam)
- Best practices: BatchNorm, Dropout, regularization

2. Convolutional Neural Networks (CNNs)

- Purpose: Handle grid-like data (images)
- Key ops: Convolution (feature detection), Pooling (downsampling)
- Modern tech: ResNet skip connections

3. Sequence Modeling (RNNs)

- Purpose: Handle sequential data (text, time series)
- Key concept: Hidden state as memory
- Advanced: LSTM/GRU gates, Encoder-Decoder for variable lengths

Key Takeaway: Right tool for the right job. CNNs for images, RNNs for sequences, MLPs for simple tasks.

Thank You!